

Aalto-yliopisto
Perustieteiden korkeakoulu
Tietotekniikan koulutusohjelma

HTML5-pelimoottorit ja web-selainten vuorovaikutusrajapinnat

Kandidaatintyö

12. huhtikuuta 2014

Sipi Seppälä

Tekijä:	Sipi Seppälä
Työn nimi:	HTML5-pelimoottorit ja web-selainten vuorovaikutusrajapinnat
Päiväys:	12. huhtikuuta 2014
Sivumäärä:	32
Pääaine:	Tietoliikenneohjelmistot
Koodi:	T3005
Vastuupettaja:	professori Juho Rousu
Työn ohjaaja(t):	professori Jukka K. Nurminen (Tietotekniikan laitos)
Tämän kandidaatintyön tavoitteena on tutustua HTML5-standardin tekniikoita käyttäviin pelimoottoreihin luokittelemalla suosituimpia pelimoottoreita niiden käyttämien grafiikkarajapintojen mukaan, sekä tutkia pelaajavuorovaikutuksen kannalta mielenkiintoisia HTML5-ohjelmointirajapintoja. Näitä ovat syötelaiteiden, kuten peliohjainten, käyttöön liittyvät sekä mobiililaitteiden erityisominaisuuksia, kuten kosketusnäyttöä ja sensoridataa, hyödyntävät rajapinnat. Lisäksi työn tavoitteena on selvittää, kuinka kattavasti nykyiset selaimet tukevat HTML5-standardia ja pohtia, riittävätkö web-selainten vakio-ominaisuudet uskottavaan ja monipuoliseen pelinkehitykseen. Pelimoottorien luokittelun aineistona käytetään listaa suosituimmista HTML5-pelimoottoreista. Pelimoottorien grafiikkatekniikoista kaksi suosituinta, Canvas-elementti ja WebGL-ohjelmointirajapinta, esitellään HTML5-standardin dokumentaation avulla. Myös vuorovaikutusrajapintojen käsittelyn ensisijaisena lähteenä käytetään HTML5-standardimäärittelyjä. Rajapintojen käyttöä peleissä tutkitaan esimerkkipelin käytännön kokeilun ja lähdekoodiin tutustumisen avulla. HTML5-selaintukea kartoitetaan rajapintojen tukitaulukolla ja peilataan tilannetta eri selainten käyttäjämäärätilastoihin. Työn tulokset kertovat, että HTML5-pelimoottoreista suurin osa on tarkoitettu ensisijaisesti kaksiulotteiselle grafiikalle, mutta tilanne voi muuttua WebGL-tuen ja erityisesti mobiililaitteiden grafiikkapiirien kehittyessä. Lisäksi työssä selvisi, että HTML5 on vuorovaikutusrajapintojen osalta täysin valmis monipuoliseksi pelialustaksi. Toisaalta havaittiin, että useiden rajapintojen selaintuki on ainakin toistaiseksi puutteellinen, mutta kehittymässä parempaan suuntaan. Siksi mahdollinen lisätutkimuskohde olisi selvittää selainpelien suorituskykyä verrattuna natiiviohjelmiin.	
Avainsanat:	HTML5, selainpelit, web-pelinkehitys, API
Kieli:	Suomi

Sisältö

1	Johdanto	4
2	Web-selain pelialustana	6
2.1	Historia	6
2.2	HTML5 ja nykyaikaiset web-sovellukset	7
2.3	HTML5-pelimoottorit	8
2.4	Pelimoottorien grafiikkarajapinnat	9
3	Web-selainten vuorovaikutusrajapinnat	11
3.1	Syötelaitteiden rajapinnat	11
3.2	Mobiililaitteille tyypilliset rajapinnat	14
3.2.1	Kosketustapahtumat	14
3.2.2	Mobiililaitteen sensorit	15
4	Rajapintojen käyttöönotto ja nykytilanne	19
4.1	HTML5:n selaintuki	19
4.2	Esimerkki rajapintojen käytöstä: WebGL Quake	23
5	Yhteenveto	27
	Lähteet	28

1 Johdanto

Verkko- eli web-selaimet ovat kehittyneet staattisten dokumenttien esittämiseen erikoistuneista lukuohjelmista monipuolisiksi vuorovaikutteellisiksi multimedia-alustoiksi, jotka toimivat monentyyppisillä laitteilla [45]. Aikaisemmin interaktiivisen sisällön tarkastelua varten piti selaimeen erikseen asentaa pitkälle erikoistuneita liitännäisiä. Nykyään selaimissa on joukko uusia ominaisuuksia ja tekniikoita, jotka toimivat suoraan ilman lisäosia. Nämä tekniikat on mahdollistanut HTML5, joka on HTML-kuvauskielen standardin uusin versio.

HTML5-termiä käytetään usein standardiin viittamisen sijaan pikemminkin yläkäsitteenä monille vielä kehitysvaiheessa oleville web-teknologioille. Keskeneräisyydestään huolimatta monet näistä teknologioista ovat jo laajasti käytössä. HTML5:n ja sen lähitekniikoiden virallisten standardien määrittelyä edistävät erityiset instituutiot, useimmiten World Wide Web Consortium (W3C) tai Web Hypertext Application Technology Working Group (WHATWG). Nämä instituutiot koostuvat joukosta vapaaehtoisia eri sidosryhmistä, aina yksittäisistä web-kehittäjistä selaimia valmistaviin yrityksiin [37, s. 3]. Joskus selainvalmistajien tuottamat toimivat toteutukset tekniikoista jopa ohjaavat standardointimäärittelyä, eikä päinvastoin.

Pelaaminen ja pelit ovat kovien alustavaatimustensa takia aina hyödyntäneet uusimpia mahdollisista teknologioista. Viime vuosiin asti web-selainpelit ovat käyttäneet erikseen asennettavia liitännäisiä [11]. Viime vuosina rajusti kasvanut selainten tuki HTML5-tekniikoille tarjoaa viimein rajapintoja monipuoliseen pelinkehitykseen suoraan selainten vakio-ominaisuuksilla, niin pöytäkoneilla kuin mobiiliselaimissakin.

Tämän kandidaatintyön tavoitteena on tutustua pelinkehityksen kannalta mielenkiintoisiin HTML5-ohjelmointirajapintoihin. Työ jakautuu kahteen aihealueeseen: HTML5-pelimoottorien luokitteluun grafiikkarajapinnan mukaan ja pelaajavuorovaikutuksen rajapintoihin.

Luvussa 2 taustoitetaan web-selaimen historiaa pelialustana, jonka jälkeen luokitellaan suosituimpia HTML5-pelimoottoreita. Luokittelun jälkeen työ esittelee tarkemmin grafiikkarajapinnoista kaksi yleisintä: *Canvas*-elementin ja *WebGL*-ohjelmointirajapinnan.

Luvussa 3 keskitytään pelaajan ja pelilaitteen välisen kommunikaation vuorovaikutusrajapintoihin. Ensiksi luku esittelee syötelaitteiden, esimerkiksi peliohjaimen ja hiiren, sekä näyttöruudun kokoruututilan rajapintoja. Tämän jälkeen käsitellään erityisesti mobiiliselaimille ominaisia rajapintoja, esimerkiksi laitteen kiihtyvysanturia, kosketusnäyttöä ja värinäominaisuutta hyödyntäviä tekniikoita.

Tämä työ keskittyy web-selaimessa ajettaviin yksinpeleihin, eikä tutki esimerkiksi

moninpelaamisen mahdollisuuksia tai tekniikoita. Tosin monet selaintekniikat toimivat pelinkehityksen kannalta samoin sekä yksin- että moninpeleissä. Myös pilvilaskentaan perustuva pelaaminen jää tämän tutkimuksen alueen ulkopuolelle.

W3C luokittelee standardointiprosessin [10] loppuvaiheessa olevat rajapinnat ja tekniikat suositusehdotuksiksi (Candidate Recommendation) ja vielä työn alla olevat luonnoksiksi (Working Draft). Valmiiksi todetut standardimäärittelyt nostetaan W3C:n suositukseksi (W3C Recommendation). Tämä työ käsittelee rajapintojen kirjoitushetkellä uusimpia versioita, jotka voivat olla joko valmiita suosituksia, suositusehdotuksia tai luonnoksia. Tässä työssä ohjelmointirajapinnasta käytetään myös englanninkielistä lyhennettä API (Application Programming Interface).

Työn tavoitteena on rajapintojen tutkimisen lisäksi selvittää HTML5:n yleistymistä, ja pohtia kuinka kypsä web-selain on pelialustana. Luvussa 4 tutkitaan rajapintojen selaintuen nykytilannetta ja pohditaan selainten suorituskykyvyn riittävyttä. Web-sovellusten suorituskyvyn ja käyttökokemuksen tavoitetasona voidaan pitää kohdealustalle erityisesti kehitettyyn ohjelmaan eli natiivisovellukseen verrattavaa tasoa [6]. Luvun lopussa tutustutaan esimerkkipelin lähdekoodin avulla vuorovaikutusrajapintojen käyttöön ja kokeillaan pelin toimivuutta eri selaimissa.

Lopuksi esitetään yhteenvetona johtopäätökset siitä, mahdollistavatko HTML5:n tarjoamat uudet ominaisuudet uskottavien ja suorituskyvyltään riittävien selainpelin kehittämisen, niin pöytäkoneympäristössä kuin mobiililaitteillakin.

2 Web-selain pelialustana

Tässä luvussa käydään läpi, millä tekniikoilla web-pelit on perinteisesti toteutettu. Sen jälkeen pohditaan, mitä HTML5-standardi tarkoittaa nykyaikaisten web-sovellusten kannalta. Lisäksi luvussa esitellään ja luokitellaan HTML5-pelimoottoreita niissä käytettyjen grafiikkarajapintojen mukaan, sillä kuvanluonnin eli grafiikan piirron toteutus on pelien kannalta olennaista. Luokittelun jälkeen tutustutaan tarkemmin pelimoottorien kahteen yleisimpään HTML5:n mukaiseen tapaan piirtää grafiikkaa: *Canvas*-elementtiin ja sitä hyödyntävään *WebGL*-grafiikkaohjelmointirajapintaan.

2.1 Historia

HTML (HyperText Markup Language) kehitettiin alun perin hypertekstidokumenttien kuvauskieleksi ja webin alkuaikoina verkkosivut olivatkin vain staattisia dokumentteja, joissa tekstin lisäksi saattoi olla liikkumattomia kuvia. [1, s. 801]

Ensimmäinen todellinen kehitysaskel kohti interaktiivista web-sisältöä oli vuonna 1996 julkaistu ECMAScript-komentosarjakieli, jonka nykyään yleisin toteutus tunnetaan nimellä JavaScript. JavaScriptin tuoma toiminnallisuus mahdollisti web-sivujen kehittymisen kohti niin sanottuja rikkaita internet-sovelluksia (Rich Internet Applications).[1, s. 801-803] Tärkeä kehitysvaihe JavaScriptin historiassa oli asynkronisen asiakas-palvelinkommunikaation mahdollistava AJAX (Asynchronous JavaScript and XML), joka on ollut tuettu selaimissa vuodesta 2005 alkaen [60, s. 1-2].

JavaScript ja AJAX eivät kuitenkaan yksin riittäneet pelien vaatimiin multimediaominaisuuksiin, ja pitkään selainpelit hyödynsivät erikseen asennettavia lisäosia. Näitä ovat esimerkiksi Java-ohjelmointikielen web-toteutukset, Java-appletit, jotka vaativat Javan ajo-ympäristön asentamisen toimiakseen. Java-tuki mahdollisti monimutkaisemman sisällön käytön selaimessa, esimerkiksi 3D-grafiikan piirtämisen, joka tosin vaatii applettien lisäksi muita Java-ympäristön osia toimiakseen. [46]

Toinen suosittu tapa toteuttaa selainpelejä oli ja on edelleenkin Adoben Flash-ympäristö, joka sekin vaatii erillisen asennuksen. Flash oli aikoinaan laajasti käytetty ja se saavutti lähes standardin aseman webin multimediasisällön toteuttamistekniikkana. Tästä huolimatta osa suurista toimijoista, erityisesti Apple, ei ollut halukas tukemaan sitä omissa laitteissaan ja selaimissaan. [11, s. 27]

Nykyään Adobe itse tarjoaa Flashin ja HTML5:n väliseksi sillaksi muun muassa Edge-kehitysympäristöä, jolla voi tehdä Flash-tyylistä multimediasisältöä HTML5-yhteensopivasti. Lisäksi Adoben julkaisema Stage3D-rajapinta tarjoaa laitteistokiihdytetyn kolmiulotteisen grafiikkarajapinnan, joka selkeästi kilpailee *WebGL*:n kanssa. [11, s. 27-29]

2.2 HTML5 ja nykyaikaiset web-sovellukset

HTML5 on HTML-kuvauskielen viides versio, ja tällä hetkellä World Wide Web Consortiumin (W3C) suositusehdokas uudeksi standardiksi [2]. Standardi on vielä keskeneräinen, mutta se edennee vahvistetuksi suositukseksi vuoden 2014 loppuun menessä [9]. HTML5-standardi on suurimmaksi osaksi taaksepäin yhteensopiva aikaisemman HTML4-version kanssa. Näin ollen se ei varsinaisesti ole edeltäjänsä korvaaja, vaan komplementoi sitä uusilla osilla. Toisaalta uusi standardi myös suosittaa web-kehittäjille joidenkin vanhentuneiksi (deprecated) merkattujen osien käyttämättä jättämistä, vaikka selaimilta niiden tukemista edelleen vaaditaankin. [44]

HTML5:llä tarkoitetaan monesti myös useita standardiin liittyviä uusia web-teknologioita [23]. HTML5 toimii siis nykyisin yleisnimityksenä ja yläkäsitteenä nykyaikaisille web-sovelluksille ja niiden käyttämille tekniikoille ja rajapinnoille. Monimutkaiseksi tilanteen tekee se, että selainvalmistajat toteuttavat selaimissaan standardia eri tavoin, ja usein valmistajilla on myös runsaasti omia rajapintamäärittelyjä. Web-sovelluskehittäjän on siis laajan alustatuen saavuttaakseen otettava huomioon eri selainten tavat ja rajat. [12]

Nykyaikainen web-sovellus koostuu yleensä kolmesta osasta: HTML-kuvauskielisestä dokumenttimallista (Document Object Model, DOM), CSS-tyylitiedostoista ja JavaScript-komentosarjoista eli skripteistä. Elementeistä koostuva HTML-dokumentti toimii alustana, jota muokkaamalla skriptit suorittavat sovelluksen toiminnallisuuden. CSS (Cascading Style Sheets) on tyylikuvauskieli, joka määrittää dokumentin tai sovelluksen ulkoasun. Vaikka CSS:n uusin versio, CSS3, sisältää paljon uusia multimediaominaisuuksia [27], ei sitä ohjelmoitavan toiminnallisuuden puutteen vuoksi voi pitää pelinkehityksen kannalta mielenkiintoisena, eikä siksi kuulu tämän tutkimuksen alueeseen.

Yksi pelien kannalta merkittävä muutos HTML5:ssä aiempiin web-tekniikoihin verrattuna on tuki laitteistokiihdytetylle 3D-grafiikalle. Vaikka Flash-ympäristöllä ja vastaavilla oli mahdollista toteuttaa kolmiulotteista kuvasisältöä, soveltuivat ne paremmin kaksiulotteisten pelien tekemiseen. HTML5:n myötä suorituskykyiset, näytönohjainta hyödyntävät kolmiulotteiset pelimaailmat on mahdollista tuoda myös web-selaimeen *WebGL*-grafiikkaohjelmoinnin avulla. [1, s. 803]

Grafiikkalaitteiston tukemisen lisäksi HTML5 tarjoaa web-kehittäjälle laajaan valikoiman muita laiterajapintoja, kuten pääsyn web-kameran kuvaan tai älypuhelimien kiihtyvyysanturin dataan. Luvussa 3 käsitellään tarkemmin näitä käyttäjävuorovaikutukseen liittyviä rajapintoja.

2.3 HTML5-pelimoottorit

Pelimoottorilla tarkoitetaan ohjelmistokehystä (application framework), joka tarjoaa pelinkehittäjälle ympäristön ja työkaluja pelien luontiin. Yleensä pelimoottori huolehtii kuvanmuodostuksesta eli renderoinnista, sekä monissa tapauksissa fysiikanmallinnuksesta. Perinteisesti pelimoottorit ovat olleet alustakohtaisia, joskin nykyään on olemassa moottoreita, jotka tukevat pelinkehitystä usealle laitealustalle yhtäaikaaisesti [11, s. 29].

HTML5-pelimoottori voidaan määritellä pelimoottorina, jonka pääasiallinen kohdealusta on web-selain ja jonka toteutus käyttää ainakin osaa HTML5-standardin tai sen lähikäsitteisiin kuuluvia teknologioita [17]. Web on luonnostaan monialustainen ympäristö, mikä antaa HTML5- ja muille web-pelimoottoreille merkittävän edun laajan pelaajakunnan saavuttamisessa. Tosin kaikki selaimet eivät toteuta HTML5-standardia samalla tavalla, mikä vaatii pelinkehitykseltä laajaa selain- ja laitekohtaista testausta [1, s. 801].

Kuten HTML5-käsitteen tarkan määrittelyn, on HTML5-pelimoottorien luokittelun tekeminen haastavaa. Ensiksi, ei olemassa virallista rajaa, missä pelimoottori ei ole enää pelimoottori vaan pelinkehityksessä hyödyllinen yleiskäyttöinen sovellus-framework tai ohjelmakirjasto. Toiseksi, pelimoottorien ja peleihin soveltuvien frameworkien määrä on valtava, eikä niiden väliset teknologiset erot ole aina ilmeisiä.

Harrastajien tekemiä listauksia kuitenkin on. Tähän tutkimukseen valittiin Clay.io-projektin ylläpitämän HTML5-pelimoottorilistan 22 pelimoottoria [15]. Taulukossa 1 nämä pelimoottorit on luokiteltu pelinäkömän ulottuvuuden ja käytetyn grafiikkarajapinnan mukaan. Ulottuvuuksien luokitteluksi valittiin 2D- ja 3D -näkömät, sekä pelimoottorit jotka tukevat sekä kaksi- että kolmiulotteista grafiikkaa. Grafiikkarajapintojen luokitteluvaihtoehdoiksi valittiin *Canvas*- tai *WebGL*-pohjaiset pelimoottorit, sekä perinteisiä HTML5-standardiin kuulumattomia web-piirtomentelmiä käyttävät pelimoottorit. *Canvas*-pohjainen pelimoottori hyödyntää HTML5-standardin kaksikulotteiselle grafikalle tarkoitettua *Canvas*-elementtiä, kun taas *WebGL*-pohjainen tarkoittaa kaksi- tai kolmiulotteista grafiikkaa *Canvas*-elementille piirtävää pelimoottoria, joka hyödyntää piirrosta *WebGL*-grafiikkaohjelmointikirjastoa. Perinteisiä piirtomenetelmiä käyttäviä pelimoottoreita merkitään taulukossa termillä HTML4.

Taulukosta 1 havaitaan, että pelkkää *Canvas*-elementtiä ilman *WebGL*-tukea käyttää puolet pelimoottoreista. Perinteisiä HTML4-pelimoottoreita, jotka piirtävät grafiikkaa muuttamalla DOM-elementtien CSS-tyylejä skripteillä pelin suorituksen aikana, löytyy listauksesta kaksi. Syy näiden listaukseen HTML5-pelimoottorien joukkoon lienee muiden HTML5-tekniikoiden kuin *Canvas*-elementin tai *WebGL*:n käyttäminen.

Taulukko 1: HTML5-pelimoottorien kuvanpiirtomenetelmät ja pelinäköymän dimensiot.

	2D	3D	2D/3D	Yht.
Canvas	11	0	0	11
WebGL	4	3	2	9
HTML4	2	0	0	2
Yhteensä	17	3	2	22

Taulukko 1 kertoo myös, että suurin osa pelimoottoreista on tarkoitettu kaksiulotteisille peleille. Yksikään pelimoottori ei toteuta kolmiulotteisuutta ilman *WebGL*-ohjelmointirajapintaa. 3D-näkymiä on kuitenkin mahdollista luoda pelkällä kaksiulotteisella *Canvas*-elementillä esimerkiksi phoria.js-ohjelmointikirjastolla [47].

Edellämainittujen lisäksi muitakin luokittelutapoja on. Juho Vepsäläisen GitHub-sivulla listataan erikseen Flash-tyyppiset tai sen inspiroimat pelimoottorit, lähdekoodiltaan pienikokoiset eli kevyet pelimoottorit, sekä kaupallisen lisenssin ohjelmakirjastot [58].

Myös tieteelliset julkaisut ovat ehdottaneet HTML5-pohjaiseen pelinkehitykseen soveltuvia sovelluskehyksiä [7] [43], joista ensimmäinen sijoittuisi taulukon luokittelussa 3D/WebGL- ja toinen 2D-kategoriaan. Tosin Penan ja Costan julkaisu [43] ei kerro, käyttäväkö heidän ohjelmakehityksensä kuvanmuodostukseen *Canvas*-elementtiä vai perinteisiä HTML4-tekniikoita.

2.4 Pelimoottorien grafiikkarajapinnat

Canvas-elementti

Canvas, tai `<canvas>`, on HTML-elementti, johon JavaScript-komentosarjat voivat piirtää bittikarttagrafiikkaa suoraan ohjelmakoodistaan. Elementti on ennen piirtoa tyhjä alue, jolle alustetaan piirto- eli renderointikonteksti. Renderointikonteksteja on tällä hetkellä kaksi: 2d ja `webgl`. 2D-kontekstia käytetään nimensä mukaisesti kaksiulotteisen grafiikan piirtoon, kun taas *WebGL*-kontekstia käyttää *WebGL*-rajapinta useimmiten 3D-grafiikalle. [22] [26]

Canvas-elementille piirto voidaan nykyselaimissa laitteistokiihdyttää eli käyttää laitteen grafiikkaprosessoria kuvanlaskentaan. Tämä tuo merkittävästi lisää suorituskykyä ja on välttämätöntä nykyaikaisilla grafiikoilla varustettujen pelien ajamiselle. [53]

Pelinkehityksen kannalta mielenkiintoinen *Canvas*-elementtiin liittyvä tekniikka on *Timing control for script-based animations* -rajapintaan kuuluva `requestAnimationFrame`-metodi, joka on skriptipohjaisten animaatioiden piirtoon suunniteltu ajastusfunktio. Skriptipohjaisissa animaatioissa ruudulle piirrettävän kuvan muutokset päivitetään

näkyviin piirtofunktiossa, jonka toistuva kutsuminen saa aikaan liikkuvan kuvan eli animaation. Piirtofunktion kutsut on perinteisesti ajastettu käyttämällä JavaScriptin yleiskäyttöisiä aikavälifunktioita, kuten `setTimeout` tai `setInterval`, pienillä aikavälin arvoilla. Näihin verrattuna `requestAnimationFrame` -kutsun etuna on se, että selaimet voivat hoitaa ja optimoida kuvanpäivityksen niin, että välittyvä animaatiokokemus on mahdollisimman hyvä. Lisäksi rajapinta mahdollistaa sen, että vain näkyvissä olevan web-sivun animaatiot piirretään, säästäten prosessorin resursseja. [48]

Ruudunpäivitysnopeudessa `requestAnimationFrame` tähtää 60 kuvaan sekunnissa [28], jota voidaan pitää riittävänä jopa kaikkein nopeatempoisiin peleihin [8].

WebGL

WebGL (*Web Graphics Library*) on *Canvas*-elementin `webgl`-kontekstia käyttävä grafiikkaohjelmointirajapinta, joka pohjautuu sulautettuihin järjestelmiin ja mobiililaitteisiin suunniteltuun OpenGL ES (OpenGL for Embedded Systems) -kirjastoon [18]. *WebGL* tukee sekä kaksi- että kolmiulotteisen grafiikan piirtämistä, mutta yleensä sitä käytetään 3D-grafiikan luontiin. *WebGL*-rajapintaa käytettäessä ohjelmointikielenä toimii JavaScript [60, s. 16], mikä poikkeaa tavallisesta OpenGL:stä, jota yleensä käytetään C-ohjelmointikielen kaltaisilla matalan abstraktiotason kielillä kuten C++:lla. Tosin *WebGL* käyttää varjostimien (shader) ohjelmointiin samaa kieltä kuin standardi-OpenGL:kin, eli GLSL:ää (OpenGL shading language). [19]

WebGL tarjoaa OpenGL:ään verrattuna web-kehittäjälle monia ohjelmointia helpottavia etuja. Monet eduista mahdollistaa `<canvas>`-elementtiin kuulumisen ja siten saavutettu tiukka integraatio dokumenttimallin kanssa. *WebGL*:ssä esimerkiksi kuvien lataaminen onnistuu käyttäen selainten valmiita kuvanlatausrajapintoja. *WebGL* voi myös hyödyntää samaa tapahtumanhallintaa (event handling) kuin mitkä tahansa muut DOM:iin liittyvät web-tekniikat. Näiden web-kehitykselle ominaisten etujen lisäksi *WebGL*:ssä on automaattinen muistinhallinta, toisin kuin OpenGL:ssä, jossa esimerkiksi varatun muistialueen vapauttaminen on ohjelmoijan vastuulla.[19] Näin ollen *WebGL*-ohjelmointi web-kehittäjälle luontevampaa JavaScriptin korkeamman abstraktiotason sekä selainten vakiorajapintojen käyttövalmiuden ansiosta. Toisaalta *WebGL* ei sisällä OpenGL:n kehittyneempiä ominaisuuksia, kuten kolmiulotteisia tekstuureja [20], mikä tarkoittaa että kolmiulotteiset selainpelit eivät ainakaan vielä voi grafiikaltaan yltää kehittyneimpien natiiviohjelmien tasolle.

3 Web-selainten vuorovaikutusrajapinnat

Tässä luvussa tutustutaan pelaajavuorovaikutusta tehostaviin rajapintoihin, joita HTML5-pelimootorit voivat käyttää valitusta grafiikkatoteutuksesta riippumatta. Esiteltäviksi rajapinnoiksi on valittu yksinpelien käyttäjävuorovaikutuksen kannalta mielenkiintoiset API:t. Työpöytäympäristössä näitä ovat syötelaitteisiin ja ikkunan kokoon liittyvät rajapinnat: ulkoisten peliohjainten käytön mahdollistava *Gamepad API*, hiiren lukitseva *Pointer Lock API* ja kokoruututilan *Fullscreen API*. Mobiiliympäristön rajapinnoista esitellään kannettavien laitteiden erityisominaisuuksia hyödyntävät tekniikat: kosketuskäyttöliittymän tapahtumat ja laitteen sensoridataa hyödyntävät rajapinnat.

W3C käyttää rajapintastandardiensa määrittelyissä WebIDL (Web Interface Definition Language) -kieltä, jossa rajapinnan erilaiset arvoja sisältävät JavaScript-objektien ominaisuudet (property) ilmaistaan termillä “attribute” [39]. Tässä luvussa näihin viitataan sanalla attribuutti.

3.1 Syötelaitteiden rajapinnat

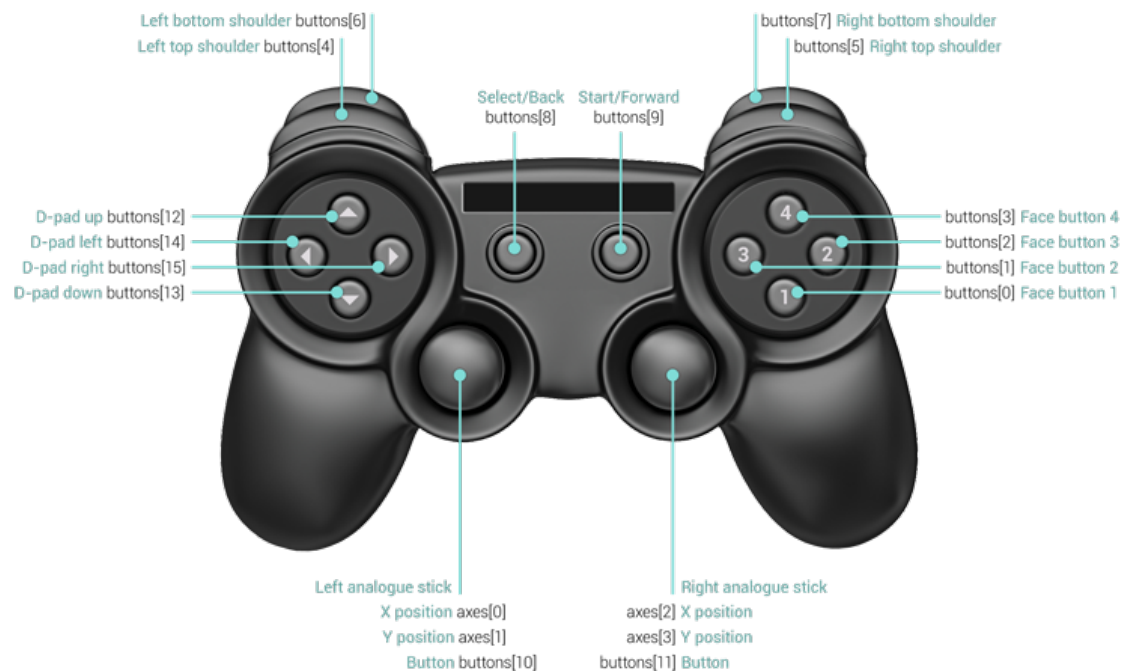
Gamepad API

Gamepad API on rajapinta, joka antaa web-ohjelmoijalle mahdollisuuden tarkastella tietokoneeseen tai laitteeseen kytkettyjä peliohjaimia. Näitä voivat olla konsoliohjainten lisäksi esimerkiksi joystickit, ratit tai polkimet. *Gamepad API*:n standardi määrittelee kahdentyyppiset syötteet: nappulat (buttons) ja akselit (axes). Näiden ainoa ero on syötteen lukuarvon alueessa: nappulat antavat arvoja välillä [0..1] ja akselit välillä [-1..1]. Nappulat voivat olla joko diskreettejä (digital) tai jatkuvia (analog). Diskreetin nappulan tapauksessa arvo-attribuutti on joko 1 tai 0, kun taas jatkuvat nappulat antavat liukuvan lukuarvon nollasta yhteen. Lisäksi nappuloilla on **pressed**-attribuutti, joka voi olla tosi tai epätosi. Selaimen vastuulla on määrittää liukuville nappuloille kynnsarvo, jonka jälkeen **pressed**-attribuutti muuttuu todeksi. [16]

Käytännössä peliohjaimia on monentyyppisiä, ja niissä on vaihteleva määrä jatkuvia ja diskreettejä painikkeita. Standardi suosittelee selaimia tunnistamaan mahdollisimman monta peliohjainta, sekä merkitsemään painikkeet mahdollisimman yhteneväksi “Standard Gamepad” -näppäinkuvauksen (kuva 1) kanssa, joka käsittää neljä akselia ja 17:ään asti nappuloita. [25]

Toisin kuin näppäimistölle ja hiirelle, *Gamepad API* ei toistaiseksi määrittele näppäinpainalluksille tapahtumia (events). Siksi web-kehittäjän tulee syötetapahtuman selvittämiseen käyttää kiertokyselyä (polling), joka yleensä toteutetaan animaation

kanssa piirtosilmukassa, esimerkiksi edellä mainittussa `requestAnimationFrame`-kutsussa.[59] Luvussa 4.2 on esimerkki kiertokyselystä.



Kuva 1: Standard Gamepad. [59]

Pointer Lock API

Pointer Lock API on hiiren liikkeisiin liittyvä rajapinta. Nimensä mukaisesti se lukitsee hiiren yhteen dokumentin elementtiin. Lisäksi se piilottaa osoittimen näkyvistä ja mahdollistaa hiiren liikkeen seuraamisen rajattoman pitkälle selaimen ja ruudun rajojen ohi, kuitenkin osoittimen ollessa lukittuna paikoilleen. [50]

Tapahutumien käsittelyn osalta *Pointer Lock* toimii kuten samankaltainen hiirenkaappaus-rajapinta *Mouse Capture API*, jossa kaikki hiiritapahtumat kohdistuvat yhteen tiettyyn kohde-elementtiin. Toisin kuin hiirenkaappauksessa, jossa hiiri vapautetaan painalluksen päättyessä, *Pointer Lock API*:ssa lukitus pysyy hiiren näppäinten tilasta riippumatta. Lukitus päättyy `exitPointerLock`-kutsulla tai kun käyttäjä tekee selainriippuvaisen poistumiseleen (unlock gesture). Poistumiseleeksi suositellaan ESC-näppäimen painallusta, josta selaimet yleensä antavat ohjeen käyttäjälle lukittumisen astuessa voimaan. [50]

Pointer Lock API on hyödyllinen erityisesti kolmiulotteisissa ensimmäisen persoonan perspektiivin peleissä, jossa pelin kameraa voi liikuttaa vapaasti. Hiiren liikkeiden osalta *Pointer Lock API* laajentaa aiempaa `MouseEvent`-rajapintaa `movementX` ja `movementY`

-attribuuteilla, joista voi lukea hiiren sijainnin muutoksen eli liikkeen. Hiiren ollessa lukittuna tämä on hyödyllisempää kuin lukea absoluuttisten sijaintikoordinaattien (attribuutit `screenX/Y` ja `clientX/Y`) arvoja, jotka lukitussa tilassa pysyvät vakioina. Liikeattribuuttien arvot ovat luettavissa myös lukitsemattomassa tilassa.[29] Näin esimerkiksi pelimaailman kameraa voi liikuttaa vaikka hiirtä ei olisi lukittu.

Fullscreen API

Osoittimen lukituksen kanssa hyvin yhteensopiva rajapinta on *Fullscreen API*, joka mahdollistaa kokoruututilan (fullscreen mode) käyttöönoton kutsumalla `requestFullscreen`-rajapintakomentoa. Toisin kuin selainten tavallisessa kokoruututilassa, *Fullscreen API*:ssa jokin dokumentin yksittäinen elementti, esimerkiksi *Canvas*-piirtoalue, asetetaan kokoruututilaan.[57] Yhdessä *Pointer Lock API*:n kanssa tämä mahdollistaa näkymältään ja käyttäjäsyötteeltään lähes natiivisovelluksen kaltaisen pelikokemuksen.

Tällä hetkellä eri selainvalmistajat toteuttavat elementin näyttämisen kokoruututilassa hieman eri tavoin. Esimerkiksi Firefoxin Gecko-piirtomoottori (layout engine) venyttää elementin leveyden ja pituuden koko ruudulle, kun taas Chromen käyttämä WebKit oletuksena keskittää elementin alkuperäisen kokoisena, jolloin ruudun reunoille jää mustana näkyvät alueet [24]. Tämän käyttäytymisen muokkaamiseen *Fullscreen API*:n standardi ehdottaa CSS-tyylin `:fullscreen` -pseudoluokkaa, jonka avulla kokoruututilassa olevan elementin näyttämisen voi muokata haluamukseen [57].

Kuten hiirilukosta, myös kokoruututilasta on käyttäjän päästävä poistumaan helposti, yleensä ESC-näppäimellä. Myös selaimen oman kokoruututila-näppäimen, tyypillisesti F11:n, painamisen sekä selaimen välilehden vaihdon suositellaan laukaisevan kokoruututilasta poistumisen. [24]

3.2 Mobiililaitteille tyypilliset rajapinnat

Mobiililaitteet, kuten tablet-tietokoneet ja älykkäät matkapuhelimet, ovat kehittyneet rajusti viime vuosina. Älypuhelimille on saatavissa mobiiliversiot yleisimmistä selaimista, eivätkä ne ole rajapinnoiltaan juurikaan puuttellisemat kuin vastaavat työpöytäversiot [12]. Prosessorien suorituskyvyn kasvamisen lisäksi mobiililaitteissa on yhä useammin erikoistuneita grafiikkapiirejä [52], joita työpöytäkoneen tavoin myös mobiiliselaimet pystyvät käyttämään.

Pelien kannalta mielenkiintoisia ovat erityisesti mobiilialustojen käyttäjäsyöterajapinnat, jotka poikkeavat melkoisesti työpöytäkoneista. Tässä luvussa esitellään kosketuskäyttöliittymien ja ympäristöä havainnoivien sensorien rajapintoja sekä kännyköiden värinäominaisuutta. Luvussa esiteltyjen rajapintojen käyttömahdollisuudet eivät toki rajoitu pelkästään mobiililaitteisiin: työpöytäkoneisiin on saatavilla esimerkiksi kosketusnäyttöjä.

3.2.1 Kosketustapahtumat

Nykyisten älypuhelimien yleisin vuorovaikutustapa on kosketuskäyttöliittymä. Web-selailussa kosketukset on perinteisesti tulkittu hiiren klikkauksiksi, mikä toimii hyvin monissa perustapauksissa, kuten nappulan tai hyperlinkin painamisessa. Monipuolisempaan vuorovaikutukseen päästään monikosketuksella (multi-touch), jossa useampi käyttäjän sormi koskettaa käyttöliittymää samanaikaisesti [34]. Peleissä monikosketukset mahdollistavat hiiritapahtumia monipuolisemman vuorovaikutuksen ilman näppäimistön käyttöä. Esimerkiksi ensimmäisen persoonan ammuntapelissä pelihahmon kontrollointi voisi tapahtua kahta sormeaa käyttämällä niin, että peliruutu on jaettu kahteen kosketusalueeseen, liikkuminen ja tähtääminen.

Monikosketuksen web-sivuilla mahdollistaa kosketustapahtumat-rajapinta, *Touch Events API* [51], joka on alun perin Applen iOS:lle kehittämä [54], mutta kuuluu nykyään W3C:n standardisuositukseseen. Kosketustapahtumien kaltainen rajapinta on osoitintapahtumat, *Pointer Events API*, jonka tavoitteena on yhdistää hiiri-, sormikosketus- ja kynäkosketustapahtumat yhdeksi monikosketustapahtumia tukevaksi rajapinnaksi [49].

Monikosketustapahtumien osalta sekä *Touch Events API* että *Pointer Events API* toimivat samankaltaisesti. Kuten myös hiiritapahtumia, kosketustapahtumia on kolme perustyyppiä: kosketuksen alkaminen, loppuminen ja liike. Jokainen kosketustapahtuma kertoo hiiritapahtumien tavoin tapahtuman sijaintikoordinaatit ja kohde-elementin dokumentissa. Lisäksi *Touch Events API* tarjoaa tapahtumalle listat `touches` ja `targetTouches`, jotka sisältävät joko koko ruutuun tai tiettyyn DOM-elementtiin kohdistuvat kosketukset, eli käytännössä luettelon koskettavista

sormista. Jokaisella sormella on uniikki tunnistenumero, `identifier`. Näiden tapahtumien ja niiden attribuuttien avulla web-kehittäjän on mahdollista luoda käsittelyt joko yleisesti tunnetuille tai mukautetuille monikosketuseleille (multi-touch gestures), joita kosketustapahtumat-rajapinta ei siis itsessään valmiiksi tue.[54] Toki mobiiliselaimet tukevat valmiiksi yleisiä navigointielementtejä, kuten rullausta ja nipistyssuurennusta (pinch zoom), jotka tapahtumienhallinnassa emuloituvat hiiren rullatapahtumiksi (wheel events) [33].

Touch Events API:n aikaisempi versio sisälsi pelien kannalta mielenkiintoinen ominaisuuden, koskettavan sormen halkaisijan koon ja kulman kuvailun [54]. Nykyisessä standardisuoitusuksessa tätä ominaisuutta ei kuitenkaan ole. Sen sijaan *Pointer Events API* tarjoaa samankaltaisia ominaisuuksia, kuten kosketuksen paineen sekä kosketuspinnan muodon (contact geometry) tunnistuksen.[49] Paineentunnistus voisi olla hyödyllinen esimerkiksi pelissä, jossa pelihahmon liikkumisnopeuteen vaikuttaa kosketuksen paineen voimakkuus.

3.2.2 Mobiililaitteen sensorit

Työpöytäkoneista poiketen mobiililaitteet keräävät yleensä ympäristöstään monenlaista sensoridataa. HTML5 tarjoaa paljon rajapintoja, joilla web-sovellukset pääsevät käyttämään tätä dataa.

DeviceOrientation Event

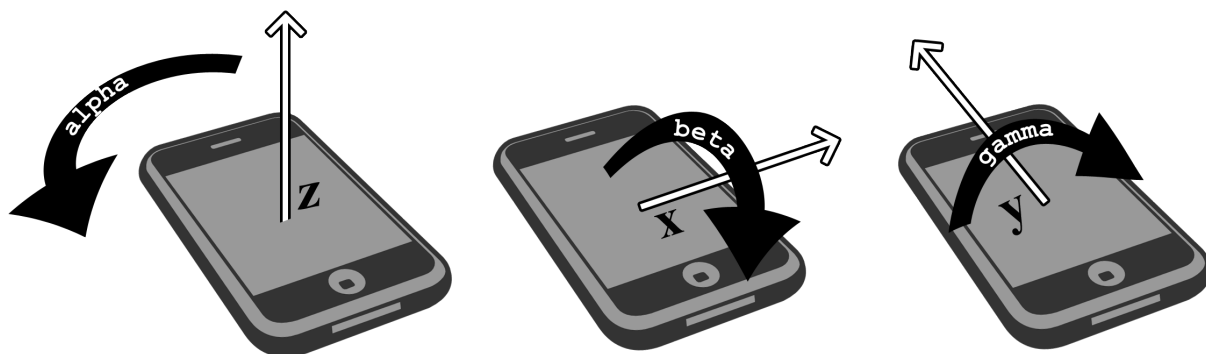
Useimpien älypuhelimien laitteistoon kuuluu gyroskooppi ja kiihtyvyysanturi sekä edellisiä magnetometrin kanssa hyödyntävä digitaalinen kompassi [32]. Näiden sensorien luomaa dataa käyttää *DeviceOrientation Event API*. Rajapinta ei tarjoa raakaa sensoridataa sellaisenaan, vaan tarkoitus on antaa sovelluksille korkeamman tason tietoa laitteen asennosta ja liikkeestä riippumatta tiedon lähteenä toimivan anturin tyypistä tai ominaisuuksista.[4] Tietoa laitteen asennosta voi peleissä käyttää esimerkiksi pelihahmon ohjaamiseen laitetta kallistamalla. Kiihtyvysdataa taas voisi hyödyntää ravistamista vaativissa tehtävissä tai esimerkiksi urheilupelien heittolajeissa.

DeviceOrientation Event -rajapinta sisältää kaksi eri tapahtumaa: laitteen asentoa kuvaava `deviceorientation` sekä liikettä kuvaava `devicemotion`. Asentotapahtuma `deviceorientation` kertoo kolmen kulman asteluvuilla laitteen kallistuksen suhteessa maahan. Maan koordinaatistossa x-akseli osoittaa itään, y-akseli pohjoiseen ja z-akseli kohtisuoraan maan pinnasta ylöspäin. Laitteen koordinaatiston x-akseli on näytön tai näppäimistön kanssa samassa tasossa ja osoittaa oikealle, kun taas y-akseli osoittaa ylös. Z-akseli laitteessa osoittaa kohtisuoraan näytöstä ulospäin. Kallistustapahtuman kolme kulma-attribuuttia `alpha`, `beta` ja `gamma` kertovat näiden koordinaatistojen välisen

kallistuksen [4]:

- **alpha**: kallistus z-akselin ympäri välillä $[0..360]$ astetta
- **beta**: kallistus x-akselin ympäri välillä $[-180..180]$ astetta
- **gamma**: kallistus y-akselin ympäri välillä $[-90..90]$ astetta

Kuva 2 havainnollistaa kallistuskulmia suhteessa pyörivään akseliin. Positiivinen kulma on akselin positiiviseen suuntaan katsottuna myötäpäivään.



Kuva 2: Laitteen kallistuskulmat akseleiden ympäri.

DeviceOrientation Event API:n toinen tapahtuma, **devicemotion**, kuvaa laitteen liikettä kahdella tapaa: kiihtyvyytenä edellä mainittujen akselien mukaisesti sekä edellä mainittujen kallistuskulmien muutosnopeutena. Attribuutti **acceleration** kertoo kunkin akselin (x, y ja z) suuntaisen kiihtyvyyden yksiköinä m/s^2 . Jotkin laitteet, joista puuttuu esimerkiksi asennon kertova gyroskooppi, eivät pysty tarjoamaan kiihtyvyydataa ilman painovoiman vaikutusta. Tällöin tieto kiihtyvyydestä annetaan **accelerationIncludingGravity**-attribuutilla, joka sisältää painovoimalle vastakkaissuuntaisen yhtäsuuren kiihtyvyyden.[4] Esimerkiksi vaakasuoraan levossa makaava laite saa **accelerationIncludingGravity**-kiihtyvyyssarvokseen $\{0, 0, 9.81\}$ eli kiihtyvyyttä z-akselin suuntaan on $9,81 \text{ m/s}^2$ [38].

Kallistuskulmien muutosnopeuden kertova attribuutti **rotationRate** antaa edellämainittujen kulmien (alpha, beta ja gamma) muutosnopeuden yksiköinä astetta/sekunti. Lisäksi **devicemotion**-tapahtuman pitää ilmoittaa **interval**-vakioarvolla millisekunteina kuinka usein kiihtyvyydataa luetaan. [4]

Vibration API

Vaikka matkapuhelimien värinäominaisuutta hyödyntävä *Vibration API* [35] ei varsinaisesti ole sensorirajapinta, on se pelien vuorovaikutuksen kannalta mielenkiintoinen. Sekä peliohjaimet että kannettavat pelikonsolit ovat jo vuosien ajan hyödyntäneet tuntoaistiin perustuvaa palautetta värisyttämällä ohjainta tai käsikonsolia pelitapahtumien mukaan. Natiiviohjelmallisissa mobiilipeleissä värinäominaisuus on niin ikään tavallista.

Vibration API:n avulla web-sovellus voi värisyttää laitetta `navigator.vibrate`-kutsulla. Kutsun argumentiksi annetaan joko värinän kesto tai lista aikamääreistä millisekuntein, joista joka toinen tarkoittaa värinäjaksoa ja joka toinen taukoa värinäjaksojen välissä. Standardin tämänhetkinen versio jättää värinäjaksojen maksimiajan sekä maksimimäärän selainvalmistajien päätettäväksi. Lisäksi se suosittaa estämään värinäominaisuuden käytön, jos ominaisuutta käyttävä web-sivu ei ole esillä, poislukien mahdollisesti etuoikeutetut sovellukset. [35]

Media Capture and Streams API

HTML5-standardin kehittyessä on kamera- ja mikrofoni- syötteiden lukemiselle ehdotettu useita rajapintoja. Näistä ensimmäinen, *Media Capture API*, on laajennus tiedostosiötteen (file input) lähettämiseen käyttäen laitteen kameraa tai mikrofonia syötteen lähteenä. Rajapinta tukee kuitenkin vain staattisia kuvia tai ennalta nauhoitettuja videotiedostoja. Tarve reaaliaikaisen kuva- ja äänisyötteen käsittelylle johti `<device>`-elementin luomiseen, mutta siitä luovuttiin nopeasti, selaintuen ollessa myöskin vähäinen.[3]

Kolmas rajapinta *MediaStream*, standardissa nimellä *Media Capture and Streams*, on tällä hetkellä suositelluin rajapinta laitteen kamera- ja mikrofoni- syötteiden kaappaamiseen [5]. Rajapinta ei ole pelkästään mobiililaitteille tyypillinen, sillä sitä voi käyttää myös esimerkiksi kannettavan tietokoneen web-kameran ja mikrofoniin kanssa.

Tärkein metodi *MediaStream API*:ssa on `getUserMedia`-funktio, jolla päästään käsiksi joko videosyötteeseen, audiosyötteeseen tai molempiin. Alla oleva koodiesimerkki pyytää laitteelta pääsyn sekä mikrofoniin että kameraan. Parametri `gotStream` on funktio, jota kutsutaan kun syötevirta alkaa. [5]

```
navigator.getUserMedia({audio:true, video:true}, gotStream);
```

MediaStream liittyy WebRTC-tiedonsiirtoprotokollaan ja on ensisijaisesti tarkoitettu käytettäväksi selainten väliseen kommunikaatioon esimerkiksi reaaliaikaisissa videokeskusteluissa. Tästä huolimatta *MediaStream*-rajapintaa voi käyttää myös paikallisesti suoritettavissa yksinpeleissä esimerkiksi pelihahmon liikuttamiseen kameran avulla.

Muut sensorirajapinnat

Muillekin mobiililaitteen sensorirajapinnoille voi löytyä innovatiivista käyttöä peleissä. Esimerkiksi käyttäjän tai esineiden läheisyyttä tunnistava *Proximity Events API* tarjoaa laitteen etäisyysanturin dataa [36]. Älypuhelimissa etäisyysanturia käytetään yleensä näyttöruudun sulkemiseen puhelimen ollessa käyttäjän korvalla [30]. Peleissä etäisyystietoa voisi käyttää esimerkiksi pelitilanteen tauottamiseen (pause) laittamalla kämmenen etäisyysanturin päälle.

Toinen ympäristön havaitsemiseen tarkoitettu rajapinta on valoisuutta havainnoiva *Ambient Light Events API*, joka kertoo `LightLevelState`-attribuutilla kuinka kirkkaasti valaistu ympäristö on. `LightLevelState` voi antaa kolme eri arvoa: himmeä (`dim`), normaali (`normal`) ja kirkas (`bright`).[56] Peleissä tätä arvoa voisi käyttää pelimaailman valaistuksen säätämiseen, esimerkiksi piirtää yömaiseman, kun ympäristön valoisuus on himmeä-tilassa.

Mikäli älypuhelimiin tulevaisuudessa tulee lisää havaitsevia sensoreita, pääsevät web-kehittäjät keksimään yhä uusia vuorovaikutuskeinoja peleihin. Tämä toki vaatii tulevia HTML-versioita ja selaimia tarjoamaan uusia rajapintoja näillekin tekniikoille.

4 Rajapintojen käyttöönotto ja nykytilanne

Tässä luvussa tarkastellaan, kuinka laajasti nykyselaimet tukevat edellämainittuja grafiikka- ja vuorovaikutusrajapintoja, sekä peilataan rajapintojen tukea selainten käyttäjämääriin. Lisäksi pohditaan, onko HTML5 valmis monipuolisten ja nykyaikaisten pelien kehittämiseksi vai puuttuuko rajapinnoista jotain pelinkehityksen kannalta oleellista. Lopuksi tutkitaan, miten WebGL Quake -esimerkkipeli käyttää edellä esiteltyjä rajapintoja, ja kokeillaan pelin toimivuutta sekä mobiili- että työpöytäselaimissa.

4.1 HTML5:n selaintuki

Taulukossa 2 on listattu suosituimpien selainten tällä hetkellä uusimpien versioiden tarjoama tuki tutkimuksessa käsitellyille rajapinnoille. Lähteenä on käytetty sekä caniuse.com-sivuston [12] että Mozilla Developer Network -oppaiden [40] yhteensopivuustaulukoita.

Taulukko 2: HTML5-rajapintojen selaintuki (maaliskuu 2014)

	Internet Explorer		Firefox		Chrome		Safari		Android Browser 4.4
	11	10(M)	28	28(M)	33	33(M)	7	7(M)	
Canvas	X	X	X	X	X	X	X	X	X
WebGL	x	-	x	x	X	X	x	-	-
Gamepad	-	-	X	-	X	-	-	-	-
Pointer Lock	-	-	X	-	X	-	-	-	-
Fullscreen	X	-	X	X	X	X	X	-	-
Touch Events	-	-	-	X	X	X	-	X	X
Pointer Events	X	x	-	-	-	-	-	-	-
DeviceOrient.	x	-	x	x	x	x	-	x	X
Vibration	-	-	X	X	X	X	-	-	X
MediaStream	-	-	X	x	X	x	-	-	-
Proximity	-	-	X	X	-	-	-	-	-
Ambient Light	-	-	x	X	-	-	-	-	-

Taulukon 2 merkintöjen selitykset:

11	työpöytäselaimen versionumero
10(M)	mobiiliselaimen versionumero
X	selain tukee rajapintaa standardin mukaisesti
x	selain tukee rajapintaa osittain tai standardista poikkeavasti
-	selain ei tue rajapintaa

Taulukosta 2 havaitaan, että grafiikkarajapinnoilla on jo laaja tuki, varsinkin *Canvas*-elementillä. Toisaalta esimerkiksi *WebGL*:n toimivuuteen ei vaikuta pelkästään käytettävän selaimen tuki, vaan myös laitteen grafiikkapiirin ominaisuudet [21]. Esimerkiksi monien mobiililaitteiden grafiikkalaitteisto ei vielä ole riittävän kehittynyt tukemaan kaikkia *WebGL*:n vaatimuksia.

Vuorovaikutus- ja syöterajapintojen vaihteleva tuki selittyy osittain sillä, että eri API:t ovat tarkoitettu erityyppisille laitteille: esimerkiksi *Pointer Lock API* on kosketusnäytöllisessä mobiililaitteessa melko hyödytön. Huomattavaa on myös samankaltaisten *Touch*- ja *Pointer Events API*:en vaihteleva selainmieltymys: Internet Explorer (IE) on ainoa selain, joka tukee osoitintapahtumia, mutta myös ainoa joka ei tue kosketustapahtumia.

Kaksi selainta, Firefox ja Chrome, ovat selkeästi muita edellä ominaisuuksien tuen kattavuudessa sekä työpöytä- että mobiiliympäristössä. Näiden osuus webin käyttäjäkunnasta on kuitenkin pieni ainakin mobiililaitteissa, joissa Safari on ylivoimaisesti suosituin noin 30-50% markkinaosuudellaan, Android-käyttöjärjestelmän oletusselaimen ollessa toiseksi suosituin noin 25%:lla. Chromen osuus mobiiliselaimista on noin 10% luokkaa, kun taas Firefoxin osuus on niin pieni ettei sitä edes mainita erikseen tilastoissa.[42][55] Android-oletusselaimen käyttäjäkunta tulee todennäköisesti kutistumaan radikaalisti, sillä Android-käyttöjärjestelmän uusimmassa 4.4-versiossa oletusselain on korvattu Chromen avoimen lähdekoodin versiolla Chromiumilla, jonka päälle puhelinvalmistajat voivat joko kehittää oman selaimen tai valita virallisen Chromen oletusselaimeksi [13].

Työpöytäselaimissa, riippuen arvion lähteestä, joko Chrome [55] tai Internet Explorer [42] on eniten käytetty selain, kattaen noin puolet kaikista käyttäjistä. Firefoxin osuus on painunut alle 20 prosentin ja ollut jo muutaman vuoden laskussa.

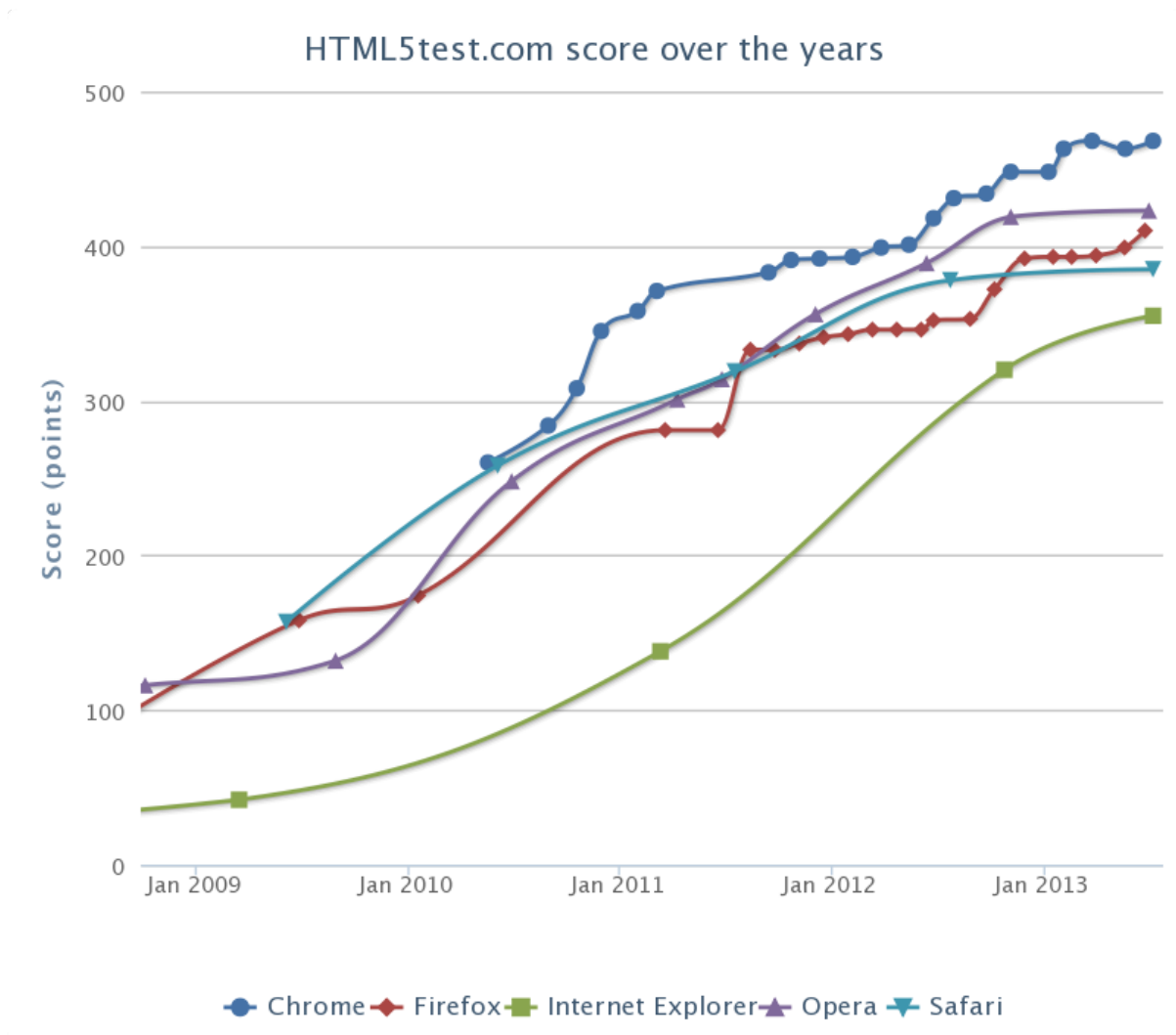
Voidaan siis todeta, että ainakin mobiiliselaimilla pelinkehittäjän, joka haluaa mahdollisimman laajan käyttäjäkunnan, on toistaiseksi tyydyttävä vanhojen rajapintojen käyttöön tai tarjottava uusia HTML5-tekniikoita vain lisäominaisuuksina, joille on vikasietoiset vanhoihin tekniikoihin perustuvat vaihtoehdot (graceful degradation). Vaihtoehto yleisten selaimien käytölle on julkaista pelejä niin kutsuttuna hybridisovelluksena, jossa peli toimitetaan natiiviohjelman tavoin, mutta

suoritetaan mukana tulevassa pelille erikseen räätälöidyssä selaimessa [61, s. 215]. Tällä erityisselaimella on luonnollisesti mahdollista käyttää rajapintoja, joita yleiskäyttöiset selaimet eivät tue.

HTML5:n selaintuen tilanne on kuitenkin muuttunut muutamassa vuodessa paljon, ja voidaan olettaa muuttuvan jatkossakin parempaan suuntaan. Vielä vuonna 2010 tehdyssä tutkimuksessa, jossa arvioitiin HTML5:n valmiutta kaikkalla läsnäolevaksi media-alustaksi, todettiin sensorituen olevan paikantamista lukuunottamatta lähes olematonta [41, s. 321]. Nykyään jo lähes kaikki, ainakin yksinpelien kannalta olennaiset, laitteistoa hyödyntävät rajapinnat on tuettu ainakin jossain yleisesti käytetyssä selaimessa. Kuvasta 3 havaitaan että suunta on jatkuvasti laajempaan HTML5-tukeen koko selainkentällä.

Rajapintatuki on tietysti vain yksi näkökulma arvioitaessa HTML5:n valmiutta pelialustaksi. Peleillä on perinteisesti ollut korkeat suorituskykyvaatimukset alustoiltaan. HTML5-tekniikoiden yleistymisen ohessa selainvalmistajat ovat käyneet kovaa kilpailua JavaScript-ohjelmien suorittamisen optimoinnista mahdollisimman nopeaksi, mikä on johtanut suuriin harppauksiin web-sovellusten suorituskyvyssä [6, s. 50-51]. JavaScriptin kiihdytyskilpailusta sekä *WebGL*:n kaltaisten matalan tason rautaa käskyttävistä rajapinnoista huolimatta HTML5 ja web-sovellukset eivät ainakaan vielä ole yltäneet natiiviohjelmien suorituskyvyn tasolle [1, s. 801], eivätkä todennäköisesti tule koskaan yltämäänsäkään [41, s. 320]. Samat suorituskykyrajoitteet koskevat myös edellä mainittuja hybridisovelluksia [61, s. 216].

Ajonaikaisen suorituskyvyn lisäksi web-sovellukset kärsivät natiiviohjelmiin verrattuna latenssiongelmasta, joka aiheutuu sovelluksen verkosta lataamiseen ja ohjelmakoodin tulkkaukseen kuluva ajasta [6, s. 52]. Toisaalta web-tekniikoiden nopeus on jo nyt riittävä moniin arkipäivän sovelluksiin. Peleille tämä saattaa tarkoittaa, että tulevaisuudessa nähdään entistä enemmän tekniikaltaan monipuolisia selainpelejä, mutta vakavamman pelaamisen tarpeen tyydyttää jatkossakin erikseen asennettavat pelisovellukset sekä tietenkin pelaamiseen ensisijaisesti erikoistuneet pelikonsolit.



Kuva 3: HTML5-tekniikoiden selaintuen kehittyminen. [14]

4.2 Esimerkki rajapintojen käytöstä: WebGL Quake

Laajasta HTML5-pelien valikoimasta tähän tutkimukseen valittiin Brandon Jonesin WebGL Quake [31]. Peli on tyypiltään ensimmäisen persoonan ammuskelupeli (first person shooter) ja tarkoitettu enemmän teknologiademoksi kuin valmiiksi tuotteeksi: pelissä ei esimerkiksi voi ampua eikä siinä ole vihollisia. Siitä huolimatta peli käyttää laajasti tässä tutkimuksessa esiteltyjä HTML5-rajapintoja. Kolmiulotteisen grafiikan pohjana on *Canvas*-elementti *WebGL*-kontekstilla. Vuorovaikutusrajapinnoista käytössä ovat *Gamepad*, *Fullscreen*, *Pointer Lock* ja *Touch Events*.

Seuraavaksi esitellään näiden vuorovaikutusrajapintojen käyttöä pelissä lähdekoodista poimituin esimerkein, joita kommentoidaan koodikatkelman jälkeen lyhyesti. Koodiesimerkit eroavat alkuperäisestä lähdekoodista siten, että koodiin on lisätty kommentteja ja siitä on poistettu osia, jotka eivät ole rajapintojen käytön kannalta mielenkiintoisia. Lisäksi pelin lähdekoodi käyttää apuna niin kutsuttua välilevykirjastoa (shim), joka muuttaa rajapintoja enemmän standardin mukaisiksi, tehden varsinaisen pelin koodista akateemisesti miellyttävämpää luettavaa.

WebGL:n alustus ja piirtosilmukan käyttö:

```
function main() {

    //alustetaan Canvas-elementille WebGL-konteksti
    var canvasElement = document.getElementById("viewport");
    var gl = getAvailableContext(canvas, ['webgl', 'experimental-webgl']);

    initEvents();           //näppäin-, kosketus- ja hiiritapahtumat
    initGL(gl, canvasElement); //alustaa WebGL:n
    renderLoop(gl, canvasElement); //kutsuu piirtosilmukkaa
}

function renderLoop(gl, element) {

    function onRequestedFrame(){

        //kutsu heti uutta animaatiokehystä
        window.requestAnimationFrame(onRequestedFrame, element);

        //kutsu syötteenkäsittelijää
        updateInput();

        //kutsu WebGL:n piirtofunktiota
        drawFrame(gl);

    }

    window.requestAnimationFrame(onRequestedFrame, element);
}
```

Kun pelin HTML-dokumentti on kokonaan latautunut, kutsutaan pelin aloittavaa

`main()`-funktioita, jossa *Canvas*-elementtiin kiinnitetään *WebGL*-konteksti. Aluksi yritetään standardin mukaista *webgl*-kontekstia, mutta yhteensopivuuden takaamiseksi annetaan vaihtoehtoiseksi kontekstiksi *experimental-webgl*, joka on toistaiseksi joissakin selaimissa ainoa mahdollisuus *WebGL*:n kokeiluun.

Syötteen lukeminen Gamepad-rajapinnalta:

```
function updateInput() {

    var dir = [0, 0, 0]; //pelaajan liikkumissuuntavektori

    //luetaan peliohjaimet taulukkoon
    var gamepads = [];
    if (navigator.getGamepads) {
        gamepads = navigator.getGamepads();
    } else if (navigator.webkitGetGamepads) {
        gamepads = navigator.webkitGetGamepads();
    }

    //käydään läpi kaikki peliohjaimet
    for (var i = 0; i < gamepads.length; ++i) {
        var pad = gamepads[i];
        if(pad) {

            //luetaan akseleista 0 ja 1 liikkumissuunta
            dir[0] += pad.axes[0];
            dir[1] -= pad.axes[1];

            //liikutetaan katsetta akselien 2 ja 3 arvojen mukaan
            moveLookLocked(pad.axes[2] * 25.0, pad.axes[3] * 25.0);

            //minkä tahansa napin painallus hyppää,
            //jos arvo on 1.0 tai pressed-attribuutti tosi
            for(var j = 0; j < pad.buttons.length; ++j) {
                var button = pad.buttons[j];
                if (typeof(button) == 'number' && button == 1.0) {
                    playerMover.jump();
                } else if (button.pressed) {
                    playerMover.jump();
                }
            }
        }
    }

    //välitä syötetiedot pelaajan liikuttajalle
    playerMover.move(dir);
}
```

Peliohjainten syötteet luetaan piirtosilmukasta toistuvasti kutsuttavassa `updateInput()`-funktiossa, mikä poikkeaa esimerkiksi kosketustapahtumista, joilla on kerran alustettavat tapahtumankuuntelijat (event listener).

Kuten *WebGL*-kontekstin tapauksessa, myös Gamepad-rajapinnan käytössä joudutaan tekemään yhteensopivuuskokeita. Standardin mukaisen

`navigator.getGamepads()`-kutsun sijaan joudutaan mahdollisesti käyttämään `navigator.webkitGetGamepads()`-funktiota.

Kun peliohjaimet on luettu taulukkoon, käydään ne läpi yksi kerrallaan. Vaikka peli ei tue kuin yhtä pelaajaa, joudutaan kaikki peliohjaimet lukemaan, sillä kytketyn peliohjaimen indeksinumeroa ei voida tietää. Nappuloilla ei myöskään esimerkissä ole muuta toiminnallisuutta kuin hyppääminen.

Kosketustapahtumien käsittely:

```
var canvasElement = document.getElementById("viewport");

//kosketus alkaa
canvasElement.addEventListener('touchstart', function(event) {
    var touches = event.touches;
    switch(touches.length) {
        case 1:
            startLook(touches[0].pageX, touches[0].pageY);
            break;
        case 2:
            startMove(touches[0].pageX, touches[0].pageY);
            break;
        case 3:
            playerMover.jump();
            break;
        default:
            return;
    }
});

//kosketuksen lopetus
canvasElement.addEventListener('touchend', function(event) {
    endLook();
    return;
});

//kosketuksen muutos
canvasElement.addEventListener('touchmove', function(event) {
    var touches = event.touches;
    switch(touches.length) {
        case 1:
            moveLook(touches[0].pageX, touches[0].pageY);
            break;
        case 2:
            moveUpdate(touches[0].pageX, touches[0].pageY, 16);
            break;
        default:
            return;
    }
});
```

Kosketustapahtumien kuuntelijoiden (event listener) alustukset ovat osa `initEvents()`-funktiota, jossa siis myös hiiri- ja näppäimistötapaukset alustetaan.

Kosketustapahtuman attribuutti `touches.length` kertoo koskettavien sormien

lukumäärän, jonka mukaan pelihahmoa ohjataan. Yhdellä sormella liikutetaan katsetta, kahdella pelihahmoa ja kolmella sormella hypätään.

Fullscreen- ja Pointer Lock -kutsut:

```
var canvasElement = document.getElementById("viewport");

//käyttäjä klikkaa kokoruututila-nappulaa
var button = document.getElementById('fullscreenBtn');
button.addEventListener('click', function() {
    canvasElement.requestFullscreen();
});

//pyydä hiiren lukitusta heti kun kokoruututila on käytössä
document.addEventListener('fullscreenchange', function() {
    if(document.fullscreenEnabled) {
        canvasElement.requestPointerLock();
    }
});

//pyydä hiirilukitusta myös heti kun käyttäjä klikkaa pelialuetta
canvasElement.addEventListener('click', function(event) {
    canvasElement.requestPointerLock();
});
```

Yllä mainitut tapahtumankuuntelijat alustetaan heti `main()`-funktiossa. Kokoruututila pyydetään pelin piirtoalueen elementille `requestFullscreen()`-kutsulla, kun käyttäjä klikkaa koko ruudun valintanappulaa. Samalla myös hiiri lukitaan `requestPointerLock()`-kutsulla, joka tehdään myös käyttäjän klikatessa pelialuetta ensimmäisen kerran.

Testiajo selaimissa

Peliä kokeiltiin Chrome- ja Firefox-selaimilla sekä työpöytäympäristössä (Sony Vaio -kannettava tietokone) että älypuhelimella (Nexus 5). Kaikissa tapauksissa peli toimi hyvin, vaikka muutaman vuoden vanhalla pöytäkoneella Firefox-selaimen ruudunpäivitys ei yltänytkaan tavoiteltuun maksimiin, 60:een kuvaan sekunnissa. Testin yllättävin tulos oli mobiiliselainten suorituskyky: sekä Chromella että Firefoxilla peli ajoi täydellä ruudunpäivitysnopeudella peli-ikkunan ollessa venytettynä koko näytölle.

Vuorovaikutusrajapinnat toimivat kokoruututilan, kosketusten ja hiiren osalta niin kuin pitikin. *Gamepad API*:n testaaminen jäi peliohjaimien puutteen vuoksi vähäiseksi. Nopeassa yhden peliohjaimen kokeilussa liikkuminen ja sivuttaissuuntainen katselu onnistui, mutta pystysuuntainen kameran liikuttaminen ei ollut mahdollista.

5 Yhteenveto

Tämän kandidaatintyön tavoitteena oli tutkia HTML5-pelimoottoreita niiden käyttämien grafiikkatekniikoiden kautta sekä nykyselainten pelinkehitykseen tarjoamia vuorovaikutusrajapintoja. Lisäksi tavoitteena oli selvittää HTML5-tuen tämänhetkistä tilannetta ja web-ympäristön valmiutta toimia vartenotettavana pelialustana.

HTML5-pelimoottorien luokittelussa havaittiin, että valtaosa suosituimmista pelimoottoreista on tarkoitettu kaksiulotteiselle grafiikalle. 2D-grafiikan suosiminen voi johtua siitä, että selainpelaaminen mielletään usein kevyemmäksi kuin vakavampaan pelaamiseen tarkoitetut pelialustat. Toisaalta tekninen tuki kolmiulotteiselle laitteistokiihdytetylle grafiikalle on vasta kypsymässä *WebGL*-rajapintaa tukevien selainten ja grafiikkapiirien yleistyessä. Molempien työssä käsiteltyjen grafiikkarajapintojen osalta selaintuen nykytilanne vaikuttaa kohtalaisen hyvältä.

Pelaajavuorovaikutusta tehostavien rajapintojen esittelyssä tutustuttiin niin perinteisten pelisyötelaitteiden kuin mobiililaitteiden uusien ominaisuuksien toimintaan web-selaimessa. HTML5-standardin kehitys kohti monipuolista vuorovaikutusalustaa on jo pitkällä, eikä juuri mitään nykypelaamiselle olennaista vuorovaikutustapaa puutu web-ympäristöstä. Käytännössä kuitenkin selaintuki monille rajapinnoille on edelleen puutteellinen. Tuen puuttumisen lisäksi haasteita luo selainvalmistajien tapa toteuttaa rajapintoja standardista poikkeavasti. Yhteensopivuusongelmista huolimatta web-pelinkehitystä voidaan pitää kohtuullisen alustariippumattomana ja siten helpompana kuin useamman natiivisovelluksen kehittämistä monille alustoille erikseen [61]. Lisäksi tulevaisuudessa tilanteen pitäisi helpottua entisestään, sillä HTML-standardoinnin päätarkoituksena on yhdenmukaistaa koko web-maailma toimimaan yhteensopivasti.

Työn viimeisenä tavoitteena oli selvittää, onko web-selain pelialustana riittävän suorituskykyinen pöytäkoneympäristön lisäksi myös mobiiliselaimissa. Rajapintojen tuen osalta mobiiliselainten voidaan todeta yltävän työpöytäversioidensa tasolle ja joidenkin mobiililaitteiden erityisominaisuuksien, kuten kosketustapahtumien, osalta jopa ylikin. Suorituskykyä taas ei niinkään rajoita selaimen mobiiliversion rajoitteellisuus, vaan suorittavan laitteiston laskentateho. Toisaalta selain on aina oma kerroksensa pelisovelluksen ja laitteiston välissä, joten natiiviohjelmien suorituskyvyn tasolle ei todennäköisesti tulla koskaan yltämään, oli kyse sitten mobiili- tai työpöytäympäristöstä.

HTML5:n standardointityön lähestyessä loppuaan lisätutkimusta voisi tehdä juurikin web-sovellusten suorituskyvystä ja selvittää, onko JavaScript-ohjelmien suoritus jo onnistuttu kiihdyttämään natiivisovellusten tasolle. Mielenkiintoista olisi nähdä, onko tulevaisuudessa mahdollista muuntaa jopa uusimmat ja raskaimmat pelit myös web-selaimessa ajettaviksi siten, että pelikokemus olisi täysin natiiviversion tasoinen.

Lähteet

- [1] M. Anttonen, A. Salminen, T. Mikkonen ja A. Taivalsaari. Transforming the web into a real application platform: New technologies, emerging trends and missing pieces. *Proceedings of the ACM Symposium on Applied Computing*, sivut 800–807, 2011. doi: 10.1145/1982185.1982357.
- [2] R. Berjon, S. Faulkner, T. Leithead, E. Doyle Navara, E. O'Connor ja S. Pfeiffer. HTML5. Candidate recommendation, World Wide Web Consortium, 2014. URL <http://www.w3.org/TR/html5>. Viitattu 27.2.2014.
- [3] E. Bidelman. Capturing Audio & Video in HTML5, 2013. URL <http://www.html5rocks.com/en/tutorials/getusermedia/intro/>. Viitattu 23.3.2014.
- [4] S. Block ja A. Popescu. DeviceOrientation Event Specification. Working draft, World Wide Web Consortium, 2011. URL <http://www.w3.org/TR/orientation-event>. Viitattu 1.3.2014.
- [5] D. C. Burnett, A. Bergkvist, C. Jennings ja A. Narayanan. Media Capture and Streams. Working draft, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/mediacapture-streams>. Viitattu 23.3.2014.
- [6] A. Charland ja B. LeRoux. Mobile application development: Web vs. native. *Communications of the ACM*, numero 5 sarjassa 54, sivut 49–53, 2011. doi: 10.1145/1941487.1941504.
- [7] B. Chen ja Z. Xu. A framework for browser-based Multiplayer Online Games using WebGL and WebSocket. *International Conference on Multimedia Technology*, sivut 471–474, 2011. doi: 10.1109/ICMT.2011.6001673.
- [8] M. Claypool, K. Claypool ja F. Damaa. The effects of frame rate and resolution on users playing first person shooter games. *Proc. SPIE*, osa 6071, 2006. doi: 10.1117/12.648609.
- [9] World Wide Web Consortium. HTML Working Group, 2014. URL <http://www.w3.org/html/wg>. Viitattu 1.3.2014.
- [10] World Wide Web Consortium. W3C Technical Report Development Process, 2014. URL <http://www.w3.org/2005/10/Process-20051014/tr>. Viitattu 7.4.2014.
- [11] K. Curran ja C. George. The Future of Web and Mobile Game Development. *International Journal of Cloud Computing and Services Science*, numero 1 sarjassa 1, sivut 25–34, 2012. doi: 10.11591/closer.v1i1.233.

- [12] A. Deveria. Caniuse - Compatibility tables for support of HTML5, CSS3, SVG and more in desktop and mobile browsers, 2014. URL <http://caniuse.com>. Viitattu 23.3.2014.
- [13] J. Dixon ja B. Murdoch. Introducing Chromium-powered Android WebView, 2013. URL <http://blog.chromium.org/2013/11/introducing-chromium-powered-android.html>. Viitattu 5.4.2014.
- [14] S. Durrani. The State Of HTML5: Today & Tomorrow!, 2013. URL <http://www.gaditek.com/blog/the-state-of-html5-today-tomorrow/>. Viitattu 28.3.2014.
- [15] HTML5 Game Engines. Which HTML5 Game Engine is right for you?, 2014. URL <http://html5gameengine.com>. Viitattu 2.3.2014.
- [16] S. Graham ja T. Mielczarek. Gamepad. Working draft, World Wide Web Consortium, 2014. URL <http://www.w3.org/TR/gamepad>. Viitattu 26.2.2014.
- [17] W. Graham. HTML5 Game Engines. *Beginning Facebook Game Apps Development*, sivut 333–354. Apress, 2012. ISBN 978-1-4302-4170-6. doi: 10.1007/978-1-4302-4171-3_13.
- [18] Khronos Group. WebGL Specification, 2014. URL <http://www.khronos.org/webgl>. Viitattu 1.3.2014.
- [19] Khronos Group. WebGL and OpenGL, WebGL Wiki, 2009. URL http://www.khronos.org/webgl/wiki/WebGL_and_OpenGL. Viitattu 20.3.2014.
- [20] Khronos Group. WebGL and OpenGL Differences, WebGL Wiki, 2009. URL http://www.khronos.org/webgl/wiki/WebGL_and_OpenGL_Differences. Viitattu 5.4.2014.
- [21] Khronos Group. BlacklistsAndWhitelists, WebGL Wiki, 2014. URL <http://www.khronos.org/webgl/wiki/BlacklistsAndWhitelists>. Viitattu 39.3.2014.
- [22] Web Hypertext Application Technology Working Group. HTML Canvas element, 2014. URL <http://www.whatwg.org/specs/web-apps/current-work/multipage/the-canvas-element.html>. Viitattu 1.3.2014.
- [23] Web Hypertext Application Technology Working Group. HTML - Living Standard , 2014. URL <http://www.whatwg.org/specs/web-apps/current-work/multipage/introduction.html>. Viitattu 1.3.2014.
- [24] MDN Web Developer Guide. Using fullscreen mode, 2014. URL http://developer.mozilla.org/en-US/docs/Web/Guide/API/DOM/Using_full_screen_mode. Viitattu 1.3.2014.

- [25] MDN Web Developer Guide. Using the Gamepad API, 2014. URL <http://developer.mozilla.org/en-US/docs/Web/Guide/API/Gamepad>. Viitattu 2.3.2014.
- [26] MDN Web Developer Guide. Canvas tutorial, 2014. URL https://developer.mozilla.org/en-US/docs/Web/Guide/HTML/Canvas_tutorial. Viitattu 2.3.2014.
- [27] MDN Web Developer Guide. CSS3, 2014. URL <https://developer.mozilla.org/en-US/docs/Web/CSS/CSS3>. Viitattu 7.4.2014.
- [28] MDN Web API Interfaces. Window.requestAnimationFrame(), 2014. URL <http://developer.mozilla.org/en/docs/Web/API/window.requestAnimationFrame>. Viitattu 1.3.2014.
- [29] MDN Web API Interfaces. Pointer Lock API, 2014. URL http://developer.mozilla.org/en-US/docs/Web/API/Pointer_Lock. Viitattu 1.3.2014.
- [30] MDN Web API Interfaces. Proximity Events, 2014. URL http://developer.mozilla.org/en-US/docs/Web/API/Proximity_Events. Viitattu 5.4.2014.
- [31] B. Jones. WebGL Quake Demo, 2013. URL <http://media.tojicode.com/q3bsp>. Viitattu 6.4.2014.
- [32] W.D. Jones. A compass in every smartphone. *Spectrum, IEEE*, numero 2 sarjassa 47, sivut 12–13, 2010. doi: 10.1109/MSPEC.2010.5397765.
- [33] G. Kacmarcik, T. Leithead, J. Rossi, D. Schepers, B. Höhrmann, P. Le Hégarret ja T. Pixley. Document Object Model (DOM) Level 3 Events Specification. Working draft, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/DOM-Level-3-Events>. Viitattu 23.3.2014.
- [34] J. Kleimola. Multimodal Interaction with HTML5, 2013. User Interface Construction -kurssin luentokalvot. Aalto-yliopisto. 6.11.2013.
- [35] A. Kostiainen. Vibration API. Working draft, World Wide Web Consortium, 2014. URL <http://www.w3.org/TR/vibration>. Viitattu 2.3.2014.
- [36] A. Kostiainen ja D. Tran. Proximity Events. Candidate recommendation, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/proximity>. Viitattu 23.3.2014.
- [37] E. Kuryanovich, S. Shalom, R. Goldenberg, M. Paumgarten, D. Strauss, S. Lee-Delisle, G. Renaudeau, J. Wagner, J. Bergknoff ja B. Danchilla. The State of Open Web Games. *HTML5 Games Most Wanted*, sivut 1–14. Apress, 2012. ISBN 978-1-4302-3978-9. doi: 10.1007/978-1-4302-3979-6_1.

- [38] P. LePage. This End Up: Using Device Orientation, 2011. URL <http://www.html5rocks.com/en/tutorials/device/orientation>. Viitattu 8.3.2014.
- [39] C. McCormack. Web IDL. Candidate recommendation, World Wide Web Consortium, 2012. URL <http://www.w3.org/TR/WebIDL>. Viitattu 5.4.2014.
- [40] Mozilla Developer Network (MDN). Web Technologies, 2014. URL <http://developer.mozilla.org/en-US/docs/Web>. Viitattu 23.3.2014.
- [41] T. Melamed ja B. Clayton. A comparative evaluation of HTML5 as a pervasive media platform. *Lecture Notes of the Institute for Computer Sciences, Social-Informatics and Telecommunications Engineering*, osa 35, sivut 307–325. LNICST, 2010. ISBN 978-3-642-12606-2. doi: 10.1007/978-3-642-12607-9_20.
- [42] netmarketshare.com. Market share statistics for internet technologies, 2014. URL <http://www.netmarketshare.com>. Viitattu 23.3.2014.
- [43] J. Pena ja C. J. Costa. Open source isometric browser games framework. *Proceedings of the Workshop - Open Source and Design of Communication*, sivut 51–53, 2010. ISBN 978-1-4503-0480-1. doi: 10.1145/1936755.1936772.
- [44] S. Pieters. Differences from HTML4. Working draft, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/html5-diff/>. Viitattu 21.3.2014.
- [45] T. V. Raman. Toward 2W, beyond web 2.0. *Communications of the ACM*, 52(2): 52–59, 2009. doi: 10.1145/1461928.1461945.
- [46] A. E. Rhalibi, M. Merabti, C. Carter, C. Dennett, S. Cooper, M. A. Sabri ja P. Fergus. 3D java web-based games development and deployment. *International Conference on Multimedia Computing and Systems*, sivut 553–559. IEEE, 2009. ISBN 978-1-4244-3756-6. doi: 10.1109/MMCS.2009.5256633.
- [47] K. Roast. phoria.js - JavaScript library for simple 3D graphics on a canvas 2D renderer, 2014. URL <http://www.kevs3d.co.uk/dev/phoria>. Viitattu 18.3.2014.
- [48] J. Robinson ja C. McCormack. Timing control for script-based animations. Candidate recommendation, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/animation-timing>. Viitattu 1.3.2014.
- [49] J. Rossi ja M. Brubeck. Pointer Events. Candidate recommendation, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/pointerevents>. Viitattu 22.3.2014.
- [50] V. Scheib. Pointer Lock. Candidate recommendation, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/pointerlock>. Viitattu 1.3.2014.

- [51] D. Schepers, S. Moon, M. Brubeck ja A. Barstow. Touch Events. W3c recommendation, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/touch-events>. Viitattu 8.3.2014.
- [52] T. Schiesser. Guide to smartphone hardware: Graphics, 2012. URL <http://www.neowin.net/news/guide-to-smartphone-hardware-27-graphics-2>. Viitattu 22.3.2014.
- [53] B. Smus. Improving HTML5 Canvas Performance, 2013. URL <http://www.html5rocks.com/en/tutorials/canvas/performance>. Viitattu 1.3.2014.
- [54] B. Smus. Multi-touch Web Development, 2011. URL <http://www.html5rocks.com/en/mobile/touch>. Viitattu 1.3.2014.
- [55] StatCounter. Global stats, 2014. URL <http://gs.statcounter.com>. Viitattu 23.3.2014.
- [56] D. Turner ja A. Kostiainen. Ambient Light Events. Candidate recommendation, World Wide Web Consortium, 2013. URL <http://www.w3.org/TR/ambient-light>. Viitattu 23.3.2014.
- [57] A. van Kesteren ja T. Çelik. Fullscreen. Working draft, World Wide Web Consortium, 2012. URL <http://www.w3.org/TR/fullscreen>. Viitattu 1.3.2014.
- [58] J. Vepsäläinen. List of HTML5 Game Engines, 2014. URL <http://github.com/bebraw/jswiki/wiki/Game-Engines>. Viitattu 27.2.2014.
- [59] M. Wichary. Jumping the Hurdles with the Gamepad API, 2012. URL <http://www.html5rocks.com/en/tutorials/doodles/gamepad>. Viitattu 2.3.2014.
- [60] J. L. Williams. *Learning HTML5 game programming: A hands-on guide to building online games using Canvas, SVG, and WebGL*. Addison-Wesley, 2012. ISBN 978-0-321-76736-3.
- [61] S. Xanthopoulos ja S. Xinogalos. A comparative analysis of cross-platform development approaches for mobile applications. *ACM International Conference Proceeding Series*, sivut 213–220, 2013. ISBN 978-1-4503-1851-8. doi: 10.1145/2490257.2490292.